Infraestrutura Categorial para *Composição de Modelos* de Aprendizado Supervisionado

Kauã Santana da Silva

Prof. Dr. Alexandre Garcia de Oliveira (orientador)

Fatec Baixada Santista “Rubens Lara”
Tecnólogo em Ciência de Dados

kaua.silva@aluno.cps.sp.gov.br

Fatec
Baixada Santista
Rubens Lara

CONIC 2026

SEÇÃO 01

Motivação

Motivação: compor algoritmos de aprendizado

Algoritmos de aprendizado podem ser representados como componentes ligados por suas entradas, saídas e operações de atualização.[1]



Cada etapa do aprendizado é um processo à parte, mas fazem parte da mesma estrutura.

Pergunta da pesquisa

Como compor algoritmos de aprendizado preservando a predição, a atualização e o fluxo de informação entre componentes?

Foco desta apresentação

Apresentar a composição sequencial e paralela de *learners*, apresentando regressão e redes neurais como exemplos práticos de modelos compostos.

[1] FONG, B.; SPIVAK, D. I.; TUYÉRAS. *Backprop as functor: a compositional perspective on supervised learning*. LICS, 2019.

CONIC 2026

SEÇÃO 02

Fundamentos

Categoria *Learn*: composição de algoritmos de aprendizado

Fong, Spivak e Tuyéras[1] formalizam algoritmos de aprendizado como *learners* na categoria *Learn*.

Ideia central do artigo

Formalizar o aprendizado de máquina supervisionado como um funtor entre categorias monoidais simétricas, mostrando que modelos complexos são meras composições de algoritmos*. A composição liga a saída de um *learner* à entrada do seguinte e descreve como a informação de aprendizado percorre os componentes.

*Compõe-se o algoritmo de aprendizado completo, não apenas sua função de predição.

O que este trabalho investiga

Estuda a formalização proposta por Fong *et al.* (2019) para a implementação de uma infraestrutura categorial para modelos de aprendizado supervisionado em Haskell via composição estrutural. Uma visão estrutural e aplicada do aprendizado supervisionado, não procedural.

[1] FONG, B.; SPIVAK, D. I.; TUYÉRAS, R. *Backprop as functor: a compositional perspective on supervised learning*. LICS, 2019.

O *learner*: unidade composicional em *Learn*

Objetos são espaços de dados; morfismos são algoritmos de aprendizado.[1]

Um morfismo em *Learn*

$f : A \to B$

Parâmetros $p \in P_f$



$U_f(p, a, b)$ atualiza parâmetros
$r_f(p, a, b)$ encaminha informação à entrada

Um *learner* reúne

Espaço de parâmetros P_f

Predição $I_f : P_f \times A \to B$

Atualização $U_f : P_f \times A \times B \to P_f$

Requisição $r_f : P_f \times A \times B \to A$

A requisição é responsável pela propagação do erro por toda estrutura.

O *backpropagation* é a regra da cadeia aplicada ao modelo

Seja $\hat{y} = (f_n \circ f_{n-1} \ldots \circ f_2 \circ f_1)(x)$ e $\mathcal{L}$ a função de perda utilizada. A perda é calculada na saída; a requisição propaga o erro pela composição, aplicando a regra da cadeia:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial f_n} \cdot \frac{\partial f_n}{\partial f_{n-1}} \cdot \ldots \cdot \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial x}$$

[1] FONG, B.; SPIVAK, D. I.; TUYÉRAS, R. (2019).

CONIC 2026

SEÇÃO 03

Exemplo prático

Exemplo na prática: rede neural em PyTorch

A arquitetura e o ciclo de otimização aparecem como instruções explícitas.[2]

Primeiro definimos a estrutura da rede: $n \to 64 \to 32 \to 1$, com *ReLU* nas camadas ocultas.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

# estrtura da rede
class Rede(nn.Module):
    def __init__(self, input_dim):
        super(Rede, self).__init__()

        # rede: n -> 64 -> 32 -> 1
        self.denseLayer1 = nn.Linear(input_dim, 64)
        self.denseLayer2 = nn.Linear(64, 32)
        self.output = nn.Linear(32, 1)

    def forward(self, x):
        x = F.relu(self.denseLayer1(x))
        x = F.relu(self.denseLayer2(x))
        x = self.output(x)
        return x
```

Perda *BCE* com *logits*, otimizador Adam, taxa de aprendizado 0,001 e ciclo de treinamento: todos à parte da estrutura. *Backpropagation* e atualização chamados explicitamente.

```python
# extracao dos dados omitida (X_train)
# normalizacao z-score (pre-treinamento)
mean    = X_train.mean(dim=0)
std     = X_train.std(dim=0)
X_train = (X_train - mean) / std

# configuracoes do modelo
modelo = Rede(input_dim=10)
bce    = nn.BCEWithLogitsLoss()
adam   = torch.optim.Adam(model.parameters(), lr=0.001)

# loop de treinamento (100 epocas)
for epoch in range(100):
    modelo.train()
    adam.zero_grad()
    saida = modelo(X_train)
    perda = bce(saida, y_train)
    perda.backward()
    adam.step()
```

[2] GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep learning*. MIT Press, 2016.

Exemplo na prática: composição em Haskell

Camadas e pré-processamento são reunidos como *learners* compostos.[1]

O operador . compõe funções da direita para a esquerda: a saída da função mais à direita alimenta a próxima função à esquerda. Inspirado no operador de composição ∘.

```haskell
-- estrutura da rede
rede mu sigma = bce . relu . denseLayer 0.3 0.0 . relu . denseLayer 0.5 0.0 . zScore mu sigma

main :: IO ()
main = do
    -- extracao dos dados omitida (trainData)
    let media  = mean trainData
        var    = std trainData
        modelo = rede media var
        p0     = iniParam modelo
        ps     = train modelo p0 trainData 100

    print ps
```

[1] FONG, B.; SPIVAK, D. I.; TUYÉRAS, R. (2019).

O que os exemplos permitem concluir?

Os códigos ilustram estilos diferentes, mas não constituem um teste comparativo controlado de eficiência computacional ou preditiva.

Python / PyTorch

Arquitetura: *Linear* $\rightarrow$ *ReLU* $\rightarrow$ *Linear* $\rightarrow$ *ReLU*
Dimensões: $n \rightarrow 64 \rightarrow 32 \rightarrow 1$
Normalização dos dados, função de perda, otimizador, taxa de aprendizado, treinamento, *backpropagation* e atualização são definidos como processos à parte de forma explicita.

Haskell / *Learner*

Arquitetura: *Z-Score* $\rightarrow$ *Linear* $\rightarrow$ *ReLU* $\rightarrow$ *Linear* $\rightarrow$ *ReLU* $\rightarrow$ *BCE*
A estrutura é declarativa, adaptando-se aos dados. Todas as transformações são encorporadas à estrutura, inlcuindo normalização, função de perda, taxa de aprendizado, *backpropagation* e atualização.

CONIC 2026

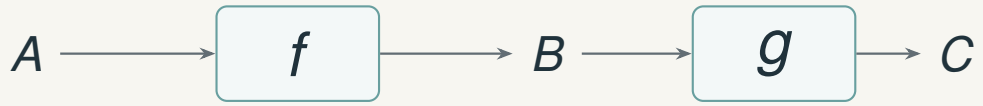
SEÇÃO 05

Composição paralela

Composição Paralela como Produto Monoidal

Sequencial encadeia saídas; paralela reúne *learners* sobre componentes distintos.[1]

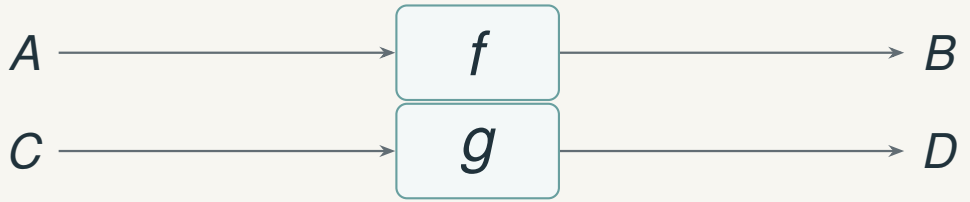
Sequencial $g \circ f$



A saída de f alimenta g.

Um *pipeline* de transformações sucessivas.

Paralela $f \parallel g$



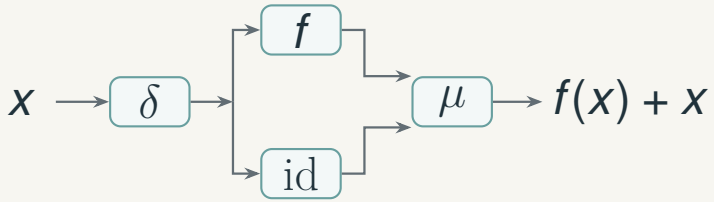
$A \times C \to B \times D$; cada ramo recebe e atualiza seus próprios dados.

Aplicações possíveis: fluxos independentes de atributos ou modalidades. Compartilhar pesos ou somar um atalho exige cópia/soma além do paralelo simples; a operação estrutural não garante execução simultânea em *hardware*.

```
residualBlock f = unsafeCoerce $ monoid . (f // id) . comonoid

residualNet mu sigma = mse . residualBlock (relu . denseLayer 0.5 0.0) .
   residualBlock (relu . denseLayer 0.3 0.0) . zScore mu sigma
```

Bloco residual



[1] FONG, B.; SPIVAK, D. I.; TUYÉRAS, R. (2019).

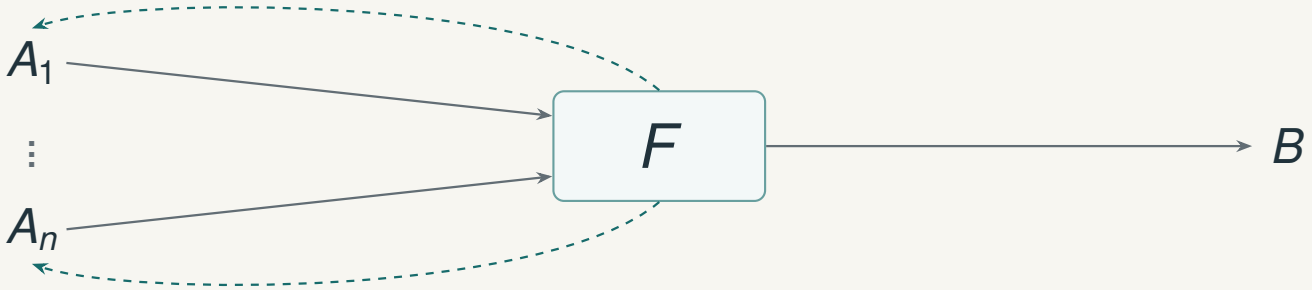
CONIC 2026

SEÇÃO 06

Extensões: multicategorias e *PROPs*

Generalização multicategorial: *MultiLearn*

Learn tem uma entrada por morfismo; *MultiLearn* generaliza para várias entradas e uma única saída.[1]




```
-- importacoes e extracao dos dados omitidos
logReg = sigmoid . linearRegressor  -- a regressao logistica como composicao da regressao linear com a sigmoid
dim    = [0.0, 0.0, 0.0, 0.0, 0.0]  -- representacao da dimensao de entrada (5), necessaria para o ajuste da estrtura

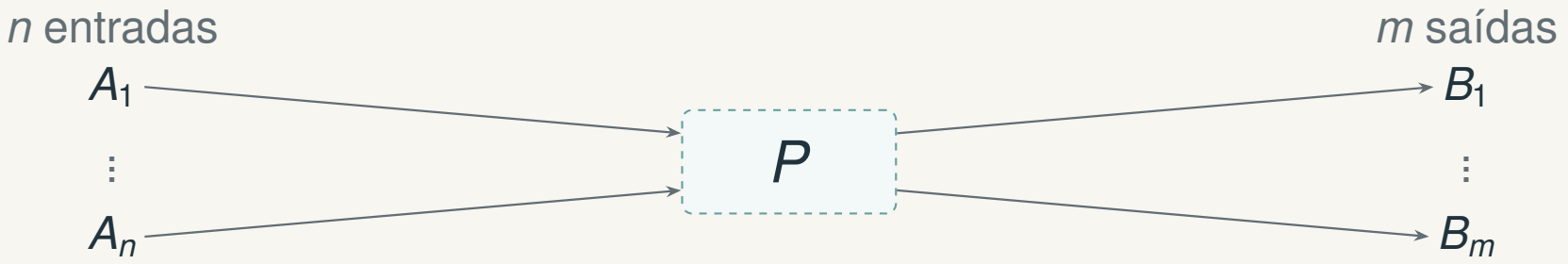
main :: IO ()
main = do
    let modelo = logReg dim
        p0     = iniParamsM modelo
        ps     = trainMulti modelo p0 trainData 100
    print ps
```

Com uma entrada, recupera-se *Learn*: a Teoria das Categorias garante que toda categoria é uma multicategoria com aridade fixada em um.[3,4]

[1] FONG *et al.* (2019); **[3]** LEINSTER (2004); **[4]** SPIVAK (2013).

Generalização para *PROPs* coloridas: *PROPsLearn*

PROPs são uma generalização das categorias, na entrada e na saída. *PROPs* coloridas admitem "cores diferentes", ou seja, tipos diferentes: *n* entradas e *m* saídas.[5]




```haskell
-- importacoes e extracao de dados omitidos
rede ws out = ccePROPsOutput out . relu . denseLayer ws
w0 k        = sin (fromIntegral (k * 97 + 131)) * 0.5            -- inicializacao nao nula (entre -0.5 e 0.5)
escondida   = [[w0 (i * 5 + j) | j <- [0..4]] | i <- [0..7]]     -- camada escondida 4 -> 8
saida       = [[w0 (40 + k * 9 + j) | j <- [0..8]] | k <- [0..2]] -- camada de saida 8 -> 3

main :: IO ()
main = do
  let modelo = rede escondida saida
      p0     = iniParamsP modelo
      ps     = trainPROPs modelo p0 trainData 200
  print ps
```

[5] LACK, S. *Composing PROPs* (2004).

CONIC 2026

SEÇÃO 07

Conclusões

Conclusões, limites e escopo

A contribuição central é compor algoritmos de aprendizado; redes neurais são exemplos práticos, não o foco do trabalho.

Contribuições

Implementa composições sequenciais e paralelas de *learners*. Os exemplos de regressão e classificação mostram a estrutura em uso sem reescrever cada algoritmo para cada conexão. *MultiLearn* e *PROPsLearn* são extensões da pesquisa e não é abordada por Fong *et al.* (2019).

Limitação da implementação

A associatividade vale matematicamente, mas o GHC não prova equivalências entre associações: a família de tipos `(++)` não é injetiva e nem sempre permite recuperar `ps` e `qs`. As projeções internas usam `unsafeCoerce`; a coerência entre os índices e o conteúdo dos GADTs fica sob responsabilidade da implementação, sem verificação do compilador.

Escopo e trabalhos relacionados, uma última observação

Escopo desta pesquisa

A pesquisa se concentra na formalização estrutural de Fong, Spivak e Tuyéras (2019). O uso de *Adam* no exemplo é contextual; não formalizamos categorialmente o otimizador.[1]

Pesquisa com escopo complementar

Cruttwell *et al.* (2022) apresentam uma semântica categorial para o aprendizado baseado em gradiente, abrangendo *Adam* e o momento de Nesterov, com lentes, mapas parametrizados e categorias de derivadas reversas.[6]

[1] FONG *et al.* (2019); **[6]** CRUTTWELL *et al.* (2022).

Referências

[1] FONG, B.; SPIVAK, D. I.; TUYÉRAS. *Backprop as functor: a compositional perspective on supervised learning*. In: LICS, 2019.

[2] GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep learning*. MIT Press, 2016.

[3] LEINSTER, T. *Higher operads, higher categories*. London Mathematical Society Lecture Note Series, 298. Cambridge University Press, 2004.

[4] SPIVAK, D. I. *The operad of wiring diagrams*. arXiv:1305.0297, 2013.

[5] LACK, S. *Composing PROPs*. *Theory and Applications of Categories*, v. 13, n. 9, p. 147–163, 2004.

[6] CRUTTWELL, G. S. H.; GAVRANOVIĆ, B.; GHANI, N.; WILSON, P.; ZANASI, F. *Categorical foundations of gradient-based learning*. In: SERGEY, I. (ed.). *Programming languages and systems (ESOP 2022)*. Lecture Notes in Computer Science, v. 13240, p. 1–28. Springer, 2022.

Obrigado!